

DSAA Lab 08

Implementation of binary tree and applying traversal techniques.

Areeba (TCT-004)

Thar Institute of Engineering Sciences and Technology (TIEST)

CT-159: Data Structure Algorithms and Applications (Pr)

Engr. Aneel Kumar

October 30, 2025

Exercise

Q1: Implement pre-order traversal using the above code.

Answer:

```
public class BinaryTreesYT {
    static class Node {
        int data;
        Node left, right;

        Node(int data) {
            this.data = data;
            left = null;
            right = null;
        }
    }

    static class BinaryTree {
        static int idx = -1;

        public static Node buildTree(int nodes[]) {
            idx++;
            if (nodes[idx] == -1) {
                return null;
            }
            Node newNode = new Node(nodes[idx]);
            newNode.left = buildTree(nodes);
            newNode.right = buildTree(nodes);
            return newNode;
        }
    }

    // 1. Preorder Traversal (Root → Left → Right)
    public static void preorder(Node root) {
        if (root == null) {
            return;
        }
        System.out.print(root.data + " ");
        preorder(root.left);
        preorder(root.right);
    }

    public static void main(String args[]) {
        int nodes[] = {1, 2, 4, -1, -1, 5, -1, -1, 3, -1, 6, -1, -1};
        BinaryTree tree = new BinaryTree();
        Node root = tree.buildTree(nodes);

        System.out.println("Preorder Traversal:");
        preorder(root);
    }
}
```

Q2: Implement post-order traversal using the above code.

Answer:

```
public class BinaryTreesYT {
    static class Node {
        int data;
        Node left, right;

        Node(int data) {
            this.data = data;
            left = null;
            right = null;
        }
    }

    static class BinaryTree {
        static int idx = -1;

        public static Node buildTree(int nodes[]) {
            idx++;
            if (nodes[idx] == -1) {
                return null;
            }
            Node newNode = new Node(nodes[idx]);
            newNode.left = buildTree(nodes);
            newNode.right = buildTree(nodes);
            return newNode;
        }
    }

    // 1. Postorder Traversal (Left → Right → Root)
    public static void postorder(Node root) {
        if (root == null) {
            return;
        }
        postorder(root.left);
        postorder(root.right);
        System.out.print(root.data + " ");
    }

    public static void main(String args[]) {
        int nodes[] = {1, 2, 4, -1, -1, 5, -1, -1, 3, -1, 6, -1, -1};
        BinaryTree tree = new BinaryTree();
        Node root = tree.buildTree(nodes);

        System.out.println("Postorder Traversal:");
        postorder(root);
    }
}
```

Q3: Modify the above program to create and traverse the binary tree based on user input.

Answer:

```
import java.util.Scanner;

// Build a Tree from its Preorder traversal
public class BinaryTreesYT {

    static class Node {
        int data;
        Node left;
        Node right;

        Node(int data) {
            this.data = data;
            this.left = null;
            this.right = null;
        }
    }

    static class BinaryTree {
        static int idx = -1;

        public static Node buildTree(int nodes[]) {
            idx++;
            if (nodes[idx] == -1) {
                return null;
            }
            Node newNode = new Node(nodes[idx]);
            newNode.left = buildTree(nodes);
            newNode.right = buildTree(nodes);
            return newNode;
        }

        public static Node buildTreeFromInput(Scanner sc) {
            System.out.print("Enter data (-1 for no node): ");
            int val = sc.nextInt();
            if (val == -1) return null;

            Node newNode = new Node(val);
            System.out.println("Enter left child of " + val);
            newNode.left = buildTreeFromInput(sc);
            System.out.println("Enter right child of " + val);
            newNode.right = buildTreeFromInput(sc);
            return newNode;
        }
    }

    // 1. Tree Traversals
    public static void preorder(Node root) {
        if (root == null) {
            System.out.print(-1 + " ");
            return;
        }
    }
}
```

```

    }
    System.out.print(root.data + " ");
    preorder(root.left);
    preorder(root.right);
}

public static void inorder(Node root) {
    if (root == null) {
        System.out.print(-1 + " ");
        return;
    }
    inorder(root.left);
    System.out.print(root.data + " ");
    inorder(root.right);
}

public static void postorder(Node root) {
    if (root == null) {
        System.out.print(-1 + " ");
        return;
    }
    postorder(root.left);
    postorder(root.right);
    System.out.print(root.data + " ");
}

// 3. Height of Tree
public static int height(Node root) {
    if (root == null) {
        return 0;
    }
    int leftHeight = height(root.left);
    int rightHeight = height(root.right);
    return Math.max(leftHeight, rightHeight) + 1;
}

// 4. Count of Nodes of Tree
public static int countOfNodes(Node root) {
    if (root == null) {
        return 0;
    }
    int leftNodes = countOfNodes(root.left);
    int rightNodes = countOfNodes(root.right);
    return leftNodes + rightNodes + 1;
}

public static void main(String args[]) {
    Scanner sc = new Scanner(System.in);
    BinaryTree tree = new BinaryTree();

    // Build tree from user input
    Node root = tree.buildTreeFromInput(sc);

    System.out.println("\nPreorder Traversal:");
    preorder(root);

    System.out.println("\nInorder Traversal:");
    inorder(root);
}

```

```
System.out.println("\nPostorder Traversal:");  
postorder(root);  
  
System.out.println("\n\nHeight of Tree: " + height(root));  
System.out.println("Count of Nodes: " + countOfNodes(root));  
}  
}
```

